Name: M. Sivateja

Batch: 41

Hall.No: 2303A52145

Task 1: Sorting Student Records for Placement Drive

PROMPT:

Write a Python program to compare Quick Sort and Merge Sort for sorting student records. Create a Student dataclass with name, roll, and cgpa. Generate random students. Sort by CGPA (descending), then name, then roll. Measure runtime using time.perf_counter().Print sorted students, performance comparison, and top 10 students.

CODE:

```python
from __future__ import annotations
import argparse
import random
import time
from dataclasses import dataclass
from typing import List, Callable
@dataclass
class Student:
    """Dataclass representing a student."""
    name: str
    roll: int
    cgpa: float
def generate_students(size: int) -> List[Student]:
    """Generate a random dataset of students with familiar names."""
    sample_names = [
        "Raju", "Rahul", "Rani", "Priya", "Sita", "Gita", "Mohan",
        "Arjun", "Anjali", "Sunita", "Vijay", "Kiran", "Asha",
        "Deepak", "Neha", "Pooja", "Rajesh", "Meena", "Suresh", "Lakshmi"
    ]
    students = []
    for i in range(size):
        name = random.choice(sample_names)
        roll = i + 1
        cgpa = round(random.uniform(5.0, 10.0), 2)
        students.append(Student(name, roll, cgpa))
    return students
def compare_students(a: Student, b: Student) -> int:
    """Comparison function for students: CGPA desc, then name asc, then roll asc."""
    if a.cgpa != b.cgpa:
        return -1 if a.cgpa > b.cgpa else 1
    if a.name != b.name:
        return -1 if a.name < b.name else 1
    return -1 if a.roll < b.roll else 1 if a.roll > b.roll else 0
```

```python
def quick_sort(arr: List[Student]) -> List[Student]:
    """Quick Sort implementation."""
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if compare_students(x, pivot) < 0]
    middle = [x for x in arr if compare_students(x, pivot) == 0]
    right = [x for x in arr if compare_students(x, pivot) > 0]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr: List[Student]) -> List[Student]:
    """Merge Sort implementation."""
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left: List[Student], right: List[Student]) -> List[Student]:
    """Helper function to merge two sorted lists."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if compare_students(left[i], right[j]) <= 0:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def measure_runtime(sort_fn: Callable[[List[Student]], List[Student]], dataset: List[Student], runs: int) -> float:
    """Measure average runtime of a sorting function."""
    total_time = 0.0
    for _ in range(runs):
        data_copy = dataset[:]
        start = time.perf_counter()
```

```python
        sorted_data = sort_fn(data_copy)
        end = time.perf_counter()
        total_time += (end - start)
        # Validate correctness
        assert sorted_data == sorted(dataset, key=lambda s: (-s.cgpa, s.name, s.roll))
    return total_time / runs
def main() -> None:
    parser = argparse.ArgumentParser(description="Compare Quick Sort and Merge Sort for student placements.")
    parser.add_argument("--size", type=int, default=1000, help="Number of students in dataset")
    parser.add_argument("--runs", type=int, default=5, help="Number of runs for averaging runtime")
    args = parser.parse_args()
    dataset = generate_students(args.size)
    quick_time = measure_runtime(quick_sort, dataset, args.runs)
    merge_time = measure_runtime(merge_sort, dataset, args.runs)
    speedup = merge_time / quick_time if quick_time > 0 else float("inf")
    print("\n============================")
    print(" Correctly Sorted Student Records ")
    print("============================")
    sorted_students = quick_sort(dataset)
    for student in sorted_students[:20]:  # show first 20 for verification
        print(f"{student.name:<8} Roll:{student.roll:<4} CGPA:{student.cgpa:.2f}")
    print("\n============================")
    print(" Performance Comparison ")
    print("============================")
    print(f"Dataset size: {args.size}, Runs: {args.runs}")
    print(f"Quick Sort: {quick_time:.6f} seconds (avg)")
    print(f"Merge Sort: {merge_time:.6f} seconds (avg)")
    print(f"Speedup (Merge/Quick): {speedup:.2f}x")
    print("\n============================")
    print(" Top 10 Students ")
    print("============================")
    for i, student in enumerate(sorted_students[:10], start=1):
        print(f"{i:2d}. Name: {student.name}, Roll: {student.roll}, CGPA: {student.cgpa:.2f}")
if __name__ == "__main__":
    main()
```

OUTPUT:

```
==============================
 Correctly Sorted Student Records
==============================
Gita     Roll:464  CGPA:9.99
Mohan    Roll:493  CGPA:9.99
Mohan    Roll:430  CGPA:9.98
Anjali   Roll:469  CGPA:9.97
Meena    Roll:130  CGPA:9.97
Raju     Roll:509  CGPA:9.97
Suresh   Roll:802  CGPA:9.95
Asha     Roll:321  CGPA:9.94
Gita     Roll:937  CGPA:9.94
Kiran    Roll:551  CGPA:9.94
Meena    Roll:165  CGPA:9.94
Sunita   Roll:819  CGPA:9.94
Kiran    Roll:278  CGPA:9.92
Mohan    Roll:352  CGPA:9.92
Sita     Roll:880  CGPA:9.92
Deepak   Roll:816  CGPA:9.91
Kiran    Roll:41   CGPA:9.91
Rani     Roll:108  CGPA:9.91
Sita     Roll:520  CGPA:9.91
Sita     Roll:963  CGPA:9.91
```

```
==============================
 Performance Comparison
==============================
Dataset size: 1000, Runs: 5
Quick Sort: 0.007389 seconds (avg)
Merge Sort: 0.002476 seconds (avg)
Speedup (Merge/Quick): 0.34x

==============================
 Top 10 Students
==============================
 1. Name: Gita, Roll: 464, CGPA: 9.99
 2. Name: Mohan, Roll: 493, CGPA: 9.99
 3. Name: Mohan, Roll: 430, CGPA: 9.98
 4. Name: Anjali, Roll: 469, CGPA: 9.97
 5. Name: Meena, Roll: 130, CGPA: 9.97
 6. Name: Raju, Roll: 509, CGPA: 9.97
 7. Name: Suresh, Roll: 802, CGPA: 9.95
 8. Name: Asha, Roll: 321, CGPA: 9.94
 9. Name: Gita, Roll: 937, CGPA: 9.94
10. Name: Kiran, Roll: 551, CGPA: 9.94
```

JUSTIFICATION:

By this code it compares Quick Sort and Merge Sort. It helps understand which algorithm performs better. Runtime measurement improves technical knowledge. It builds practical experience with data structures.

Task 2: Implementing Bubble Sort with AI Comments

PROMPT:

Write a simple Python Bubble Sort program with user input, inline comments explaining swapping, passes, and termination, and include time complexity analysis.

CODE:

```python
#Write a simple Python Bubble Sort program with user input, inline comments explaining swapping, passes, and termination, and include time comple
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]  # Swapping
                swapped = True  # Set flag to True if swapping occurs
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break  # Terminate early if the array is sorted
    return arr
# Get user input for the list of numbers to sort
input_str = input("Enter numbers to sort, separated by commas: ")
# Convert the input string into a list of integers
numbers = list(map(int, input_str.split(',')))
# Sort the numbers using bubble sort
sorted_numbers = bubble_sort(numbers)
# Print the sorted list
print("Sorted numbers:", sorted_numbers)
# Time Complexity Analysis:
# The time complexity of Bubble Sort is O(n^2) in the worst and average cases, where n is the number of elements in the array. This is because in
# In the best case, when the array is already sorted, the time complexity is O(n) due to the optimization that checks for swapping. The space com
```

OUTPUT

```
Enter numbers to sort, separated by commas: 5,3,13,2,8,9
Sorted numbers: [2, 3, 5, 8, 9, 13]
```

JUSTIFICATION:

I implemented Bubble Sort to understand basic sorting logic. It helped me learn how swapping works step by step. Adding comments improved code clarity. This task strengthened my programming basics.

Task 3: Quick Sort and Merge Sort Comparison

PROMPT:

Complete partially written recursive functions for Quick Sort and Merge Sort in Python. Add proper docstrings and implement the missing logic. Then compare both algorithms on random, sorted, and reverse-sorted lists, measure runtime, and provide explanation of best, average, and worst-case time complexities.

CODE:

```python
#Generate recursive Quick Sort and Merge Sort in Python with docstrings, runtime comparison on different list types, and complexity analysis.
import random
import time
# --------------- QUICK SORT ---------------
def quick_sort(arr):
    """
    Recursive Quick Sort implementation.
    Divides the list into smaller sublists based on a pivot element.
    Returns a new sorted list.
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]  # choose middle element as pivot
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
# --------------- MERGE SORT ---------------
def merge_sort(arr):
    """
    Recursive Merge Sort implementation.
    Divides the list into halves, sorts each half,
    and merges them back together.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

```python
def merge(left, right):
    """Helper function to merge two sorted lists."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
# ---------------- TESTING ----------------

def test_sorting(data, name):
    print(f"\n{name} List Test")
    start = time.perf_counter()
    quick_sort(data)
    quick_time = time.perf_counter() - start
    start = time.perf_counter()
    merge_sort(data)
    merge_time = time.perf_counter() - start
    print("Quick Sort Time:", round(quick_time, 6))
    print("Merge Sort Time:", round(merge_time, 6))
# Random list
random_list = random.sample(range(1, 10000), 1000)
```

```python
# Random list
random_list = random.sample(range(1, 10000), 1000)
# Sorted list
sorted_list = sorted(random_list)
# Reverse sorted list
reverse_list = sorted(random_list, reverse=True)
test_sorting(random_list, "Random")
test_sorting(sorted_list, "Sorted")
test_sorting(reverse_list, "Reverse Sorted")
```

OUTPUT:

```
Random List Test
Quick Sort Time: 0.003039
Merge Sort Time: 0.006066

Sorted List Test
Quick Sort Time: 0.002448
Merge Sort Time: 0.004027

Reverse Sorted List Test
Quick Sort Time: 0.004525
Merge Sort Time: 0.004501
PS C:\Users\sdnaz>
```

EXPLANTION:

```python
#AI-generated explanation of average, best, and worst-case complexities.
"""
Time Complexity Analysis:
- Quick Sort:
  - Average Case: O(n log n) - occurs when the pivot divides the list into two equal halves.
  - Best Case: O(n log n) - occurs when the pivot is consistently the median.
  - Worst Case: O(n^2) - occurs when the smallest or largest element is always chosen as the pivot (e.g., already sorted or reverse sorted lists).
- Merge Sort:


    - Average Case: O(n log n) - occurs for all cases since it always divides the list into halves.
    - Best Case: O(n log n) - occurs for all cases since it always divides the list into halves.
    - Worst Case: O(n log n) - occurs for all cases since it always divides the list into halves.
Space Complexity:
- Quick Sort: O(log n) on average due to recursive stack space, but O(n) in the worst case.
- Merge Sort: O(n) due to the temporary arrays used for merging.
"""
```

JUSTIFICATION:

This task helped me understand recursive sorting algorithms. I learned how Quick Sort and Merge Sort divide problems into smaller parts. Comparing different input types improved my analysis skills. It strengthened my knowledge of recursion and efficiency.

Task 4 (Real-Time Application – Inventory Management System)

PROMPT:

Generate a Python Inventory Management System that recommends efficient search and sorting algorithms, implements them, and displays the expected output clearly.

CODE:

```python
# ---------------- INVENTORY DATA ----------------
inventory = [
    {"id": 101, "name": "Laptop", "price": 55000, "quantity": 10},
    {"id": 102, "name": "Mouse", "price": 500, "quantity": 100},
    {"id": 103, "name": "Keyboard", "price": 1500, "quantity": 50},
    {"id": 104, "name": "Monitor", "price": 12000, "quantity": 20},
    {"id": 105, "name": "Printer", "price": 8000, "quantity": 15},
]
# Create dictionary for fast ID search
inventory_by_id = {product["id"]: product for product in inventory}
# ---------------- SEARCH FUNCTIONS ----------------
def search_by_id(product_id):
    """Search product using dictionary (O(1))"""
    return inventory_by_id.get(product_id, "Product not found")
def search_by_name(name):
    """Search product by name (linear search)"""
    for product in inventory:
        if product["name"].lower() == name.lower():
            return product
    return "Product not found"
# ---------------- SORT FUNCTIONS ----------------
def sort_by_price():
    """Sort inventory by price (ascending)"""
    return sorted(inventory, key=lambda x: x["price"])
def sort_by_quantity():
    """Sort inventory by quantity (ascending)"""
    return sorted(inventory, key=lambda x: x["quantity"])
# ---------------- TESTING ----------------
print("Search by ID (102):", search_by_id(102))
print("Search by Name (Monitor):", search_by_name("Monitor"))
print("\nSorted by Price:")
for item in sort_by_price():
    print(item)
print("\nSorted by Quantity:")
for item in sort_by_quantity():
    print(item)
```

OUTPUT:

```
Search by ID (102): {'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 100}
Search by Name (Monitor): {'id': 104, 'name': 'Monitor', 'price': 12000, 'quantity': 20}

Sorted by Price:
{'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 100}
{'id': 103, 'name': 'Keyboard', 'price': 1500, 'quantity': 50}
{'id': 105, 'name': 'Printer', 'price': 8000, 'quantity': 15}
{'id': 104, 'name': 'Monitor', 'price': 12000, 'quantity': 20}
{'id': 101, 'name': 'Laptop', 'price': 55000, 'quantity': 10}

Sorted by Quantity:
{'id': 101, 'name': 'Laptop', 'price': 55000, 'quantity': 10}
{'id': 105, 'name': 'Printer', 'price': 8000, 'quantity': 15}
{'id': 104, 'name': 'Monitor', 'price': 12000, 'quantity': 20}
{'id': 103, 'name': 'Keyboard', 'price': 1500, 'quantity': 50}
{'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 100}
PS C:\Users\sdnaz>
```

Recommended Algorithms Table

Operation	Recommended Algorithm	Justification
Search by Product ID	Hash Table (Dictionary)	Fast lookup O(1), best for large datasets and frequent searches
Search by Product Name	Linear Search (or Dictionary if indexed)	Simple and effective; can also use dictionary for faster lookup
Sort by Price	Timsort (Python built-in sorted())	Very efficient O(n log n), optimized for real-world data
Sort by Quantity	Timsort (Python built-in sorted())	Stable, fast, and handles large datasets efficiently

JUSTIFICATION:

This task helps design a real-time inventory system. Using efficient search improves product lookup speed. Sorting helps analyze stock and pricing easily. It applies algorithms to a real-world business scenario.

Task 5: Real-Time Stock Data Sorting & Searching

PROMPT:

Create a menu-driven Python program to simulate stock data, sort stocks by percentage change using Heap Sort, search stocks using a Hash Map (dictionary), compare performance with built-in sorted(), and display the output clearly.

CODE:

```
import random
import time
def generate_stocks(n):
    symbols=["TCS","INFY","HDFC","RELIANCE","WIPRO","SBIN","ITC","ADANI","AXIS","ICICI"]
    stocks=[]
    for i in range(n):
        symbol=random.choice(symbols)+str(i)
        open_price=random.randint(100,5000)
        close_price=open_price+random.randint(-200,200)
        percent_change=((close_price-open_price)/open_price)*100
        stocks.append({"symbol":symbol,"open":open_price,"close":close_price,"change":round(percent_change,2)})
    return stocks
def heapify(arr,n,i):
    largest=i
    left=2*i+1
    right=2*i+2
    if left<n and arr[left]["change"]>arr[largest]["change"]:
        largest=left
    if right<n and arr[right]["change"]>arr[largest]["change"]:
        largest=right
    if largest!=i:
        arr[i],arr[largest]=arr[largest],arr[i]
        heapify(arr,n,largest)
```

```python
def heap_sort(arr):
    n=len(arr)
    for i in range(n//2-1,-1,-1):
        heapify(arr,n,i)
    for i in range(n-1,0,-1):
        arr[i],arr[0]=arr[0],arr[i]
        heapify(arr,i,0)
    return arr[::-1]

def build_hash_map(stocks):
    return {stock["symbol"]:stock for stock in stocks}

def search_stock(stock_map,symbol):
    return stock_map.get(symbol,"Stock not found")

# Generate data
n=1000
stocks=generate_stocks(n)
stock_map=build_hash_map(stocks)
sorted_heap=heap_sort(stocks.copy())

while True:
    print("\n--- Stock Analysis Menu ---")
    print("1. Show Top 5 Stocks")
    print("2. Search Stock by Symbol")
    print("3. Performance Comparison")
    print("4. Exit")

    choice=input("Enter your choice: ")

    if choice=="1":
        print("\nTop 5 Stocks by % Gain/Loss:\n")
        for i in range(5):
            s=sorted_heap[i]
            print(f"{i+1}. {s['symbol']} | Open:{s['open']} | Close:{s['close']} | Change:{s['change']}%")
```

```python
    elif choice=="2":
        symbol=input("Enter Stock Symbol: ")
        result=search_stock(stock_map,symbol)
        print("Search Result:",result)

    elif choice=="3":
        start=time.perf_counter()
        heap_sort(stocks.copy())
        heap_time=time.perf_counter()-start

        start=time.perf_counter()
        sorted(stocks,key=lambda x:x["change"],reverse=True)
        builtin_time=time.perf_counter()-start

        print("\nPerformance Comparison:")
        print("Heap Sort Time:",round(heap_time,6),"seconds")
        print("Built-in sorted() Time:",round(builtin_time,6),"seconds")

    elif choice=="4":
        print("Exiting program...")
        break

    else:
        print("Invalid choice! Try again.")
```

OUTPUT:

```
--- Stock Analysis Menu ---
1. Show Top 5 Stocks
2. Search Stock by Symbol
3. Performance Comparison
4. Exit
Enter your choice: 1

Top 5 Stocks by % Gain/Loss:

1. INFY68 | Open:130 | Close:299 | Change:130.0%
2. TCS627 | Open:110 | Close:227 | Change:106.36%
3. HDFC561 | Open:124 | Close:242 | Change:95.16%
4. WIPRO345 | Open:111 | Close:207 | Change:86.49%
5. HDFC815 | Open:212 | Close:392 | Change:84.91%

--- Stock Analysis Menu ---
1. Show Top 5 Stocks
2. Search Stock by Symbol
3. Performance Comparison
4. Exit
Enter your choice: 2
Enter Stock Symbol: INFY45
Search Result: Stock not found
```

```
--- Stock Analysis Menu ---
1. Show Top 5 Stocks
2. Search Stock by Symbol
3. Performance Comparison
4. Exit
Enter your choice: 3

Performance Comparison:
Heap Sort Time: 0.004049 seconds
Built-in sorted() Time: 0.000281 seconds

--- Stock Analysis Menu ---
1. Show Top 5 Stocks
2. Search Stock by Symbol
3. Performance Comparison
4. Exit
Enter your choice: 4
Exiting program...
PS C:\Users\sdnaz>
```

JUSTIFICATION:

This project simulates real-time stock market analysis. Heap Sort helps rank stocks efficiently by percentage change. Hash Map enables instant stock symbol search. It improves understanding of data handling in FinTech systems.